Intelligent UPI Fraud Detection System Using Machine Learning

Mini project report submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Under the esteemed guidance of

Dr. V. Shiva Narayana Reddy

Associate Professor

By

Gurnule Snehal (23U11A0564)

Department of Computer Science and Engineering

Samskruti College of Engineering and Technology (Autonomous)

Approved by AICTE & Affiliated to JNTUH, Accredited by NBA and NAAC with A Grade

Accredited by NBA Kondapur(V), Ghatkesar (M), Medchal District, Telangana — 501301

June — 2026

Samskruti College of Engineering and Technology (Autonomous)

Approved by AICTE & Affiliated to JNTUH, Accredited by NBA and NAAC with A Grade Kondapur(V), Ghatkesar (M), Medchal District, Telangana — 501301

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the B.Tech Mini Project report entitled **"Intelligent UPI Fraud Detection System Using Machine Learning "** is a bonafide work done by **Gurnule Snehal (23U11A0564),** in partial fulfillment of the requirement of the award for the degree of Bachelor of Technology in "**Computer Science and Engineering**" from Jawaharlal Nehru Technological University, Hyderabad during the year 2025-2026.

Dr. V. Shiva Narayana Reddy
Associate Professor

HOD — CSE
Dr. V. Shiva Narayana Reddy
Associate Professor

Examiner

Samskruti College of Engineering and Technology (Autonomous)

Approved by AICTE & Affiliated to JNTUH, Accredited by NBA and NAAC with A Grade

Kondapur(V), Ghatkesar (M), Medchal District, Telangana — 501301

Department of Computer Science and Engineering

DECLARATION BY THE CANDIDATE

I, **Gurnule Snehal ,** bearing Roll No. **23U11A0564** hereby declare that the project report entitled **"Intelligent UPI Fraud Detection System Using Machine Learning"** is done under the guidance of **Dr. V. Shiva Narayana Reddy, Associate Professor**, Department of Computer Science and Engineering, Samskruti College of Engineering and Technology, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering.**

This is a record of bonafide work carried out by me and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other University or Institute for the award of any other degree or diploma.

Gurnule Snehal, 23U11A0564,

Department of CSE,
Samskruti College of Engineering and Technology,
Ghatkesar.

ACKNOWLEDGEMENT

It is with profound gratitude and sincere appreciation that I extend my heartfelt thanks to all those who have played a significant role in the successful completion of this undergraduate project.

First and foremost, I express my deep sense of respect and gratitude to our **Honourable Chairman, Mr. A. V. Ramana Reddy,** for his constant encouragement and for nurturing a culture of academic excellence and innovation within the institution.

I would also like to express my sincere thanks to **Dr. M. Ramakanth Reddy,** Director, for his visionary leadership and continued support, which have provided the ideal environment and motivation for academic pursuits such as this project.

My heartfelt appreciation goes to **Dr. P. Janaki Ramulu, Principal,** for his steadfast guidance, infrastructural support, and encouragement that have helped bring this project to successful fruition.

I am deeply grateful to **Dr. V. Shiva Narayana Reddy, Head of the Department of Computer Science and Engineering,** for his academic leadership, valuable feedback, and continuous support throughout the duration of this project.

A special note of gratitude is reserved for my project guide, **Dr. V. Shiva Narayana Reddy, Associate Professor,** whose expert supervision, insightful suggestions, and dedicated mentorship have been instrumental in shaping the direction and outcome of this work.

Lastly, I am ever grateful to my parents and family for their unconditional love, encouragement, and moral support. Their faith in me has been my greatest strength throughout this journey.

With genuine appreciation, I acknowledge every individual who has, in one way or another, contributed to the successful completion of this project.

With warm regards,

Gurnule Snehal (23U11A0564)

Intelligent UPI Fraud Detection System Using Machine Learning

ABSTRACT

UPI (Unified Payments Interface) has made digital payments incredibly fast and convenient across India, but this rapid growth has also led to a sharp rise in online payment fraud. Fraudsters use tricks like fake QR codes, phishing links, and OTP manipulation to steal money from innocent users. Traditional rule-based fraud detection systems are not smart enough to catch these ever-changing fraud patterns, which is why there is a need for a more intelligent and adaptive solution. In this project, we propose a machine learningbased system to automatically detect fraudulent UPI transactions in real time. We used transaction data containing features like amount, time, location, device information, and user behavior to train popular ML models such as Logistic Regression, Random Forest, and XGBoost. Since fraud cases are much fewer than normal transactions, we used a technique called SMOTE (Synthetic Minority Oversampling Technique) to balance the dataset and improve the model's ability to detect fraud accurately.

Our experimental results show that the proposed system achieves an accuracy of over 95%, with high precision and recall values, meaning it catches most fraudulent transactions while keeping false alarms low. The model is capable of flagging suspicious transactions in real time, making it practical for deployment in real-world payment systems. Compared to traditional approaches, our ML-based solution is more scalable, adaptive, and effective in protecting UPI users from financial fraud.

LIST OF FIGURES/DIAGRAMS/SCREENSHOTS

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
Python	Python Programming Language
Django	Django Web Framework
SQL	Structured Query Language
SQLite	Structured Query Language Lite
CSV	Comma-Separated Values
API	Application Programming Interface
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
RF	Random Forest

Table of Content

CHAPTER – 1

OVERVIEW OF THE PROJECT

1.1 Introduction

The rapid growth of digital payment systems and the increasing adoption of Unified Payments Interface (UPI) have transformed the financial transaction landscape into one of the fastest-growing sectors of the digital economy. As the number of online transactions continues to increase, ensuring transaction security and preventing fraudulent activities have become major challenges for banks, payment service providers, and users. Fraudulent practices such as phishing attacks, identity theft, fake payment requests, account takeovers, and unauthorized transactions pose significant risks to both customers and financial institutions. Traditional fraud detection methods rely heavily on predefined rules, manual monitoring, and expert judgment, which are often ineffective in identifying sophisticated and evolving fraud patterns. These approaches frequently result in delayed fraud detection, increased financial losses, and reduced trust in digital payment platforms. To address these limitations, this project titled "Intelligent UPI Fraud Detection System Using Machine Learning" introduces an intelligent web-based fraud detection platform that utilizes Machine Learning algorithms to identify suspicious transactions automatically and accurately. The proposed system combines predictive analytics, machine learning, modern web technologies, and interactive dashboards to create a comprehensive fraud detection and transaction monitoring ecosystem.

1.2 Objective of the Project

The primary objective of this project is to develop an intelligent web-based application capable of detecting fraudulent UPI transactions using Machine Learning techniques. The system aims to provide accurate and reliable fraud detection by analyzing historical transaction data and identifying relationships between important transaction attributes such as transaction amount, transaction frequency, transaction location, device information, transaction time, and user behavior patterns. Another objective is to reduce dependence on traditional rule-based fraud detection methods and minimize financial risks associated with online payment fraud. The project also focuses on developing a responsive and user-friendly interface that allows users to securely register, log in, monitor transactions, and receive instant fraud prediction results. Additionally, the system aims to integrate dashboard analytics for visualizing fraud detection outcomes, transaction trends, and security insights. Administrative functionalities are included to monitor system

performance, manage users, analyze transaction records, generate reports, and maintain historical fraud prediction data. The overall goal is to enhance the security, reliability, and trustworthiness of UPI-based digital payment systems through intelligent fraud detection mechanisms.

1.3 Scope of the Project

The scope of the Intelligent UPI Fraud Detection System Using Machine Learning is to provide a secure and intelligent platform for detecting fraudulent UPI transactions and enhancing the safety of digital payment systems. The project focuses on analyzing transaction data using Machine Learning algorithms to identify suspicious activities and predict potential fraud based on transaction attributes such as amount, frequency, location, transaction time, and user behavior patterns. The system provides functionalities for user registration, authentication, transaction monitoring, fraud prediction, and administrative management through a web-based interface. It enables users to securely access the platform and receive information regarding potentially fraudulent transactions, while administrators can monitor fraud statistics, manage users, review transaction records, and analyze system performance through dashboards and reports. Furthermore, the system can be extended with advanced Machine Learning techniques, real-time transaction processing, mobile application support, and integration with banking systems. The project can be utilized by banks, financial institutions, and digital payment service providers to strengthen transaction security, reduce financial losses, and increase user confidence in digital payment services, thereby contributing to a safer and more reliable UPI payment ecosystem.

CHAPTER – 2

LITERATURE SURVEY

2.1 Existing Technologies

Existing fraud detection technologies in digital payment systems have led to the development of various security solutions, banking applications, and machine learning–based approaches for identifying fraudulent transactions. Several financial institutions and payment service providers have implemented automated systems to monitor transaction activities and detect suspicious behavior. These systems aim to reduce financial fraud, improve transaction security, and assist organizations in protecting users from unauthorized activities. However, the methodologies, implementation techniques, and detection capabilities vary significantly across different platforms and solutions. One category of existing technologies includes traditional rule-based fraud detection systems used by banks and digital payment providers. These systems rely on predefined rules, transaction thresholds, location-based verification, device authentication, and historical transaction records to identify potentially fraudulent activities. When unusual transaction patterns are detected, alerts are generated for further investigation. Although these systems provide a basic level of security and help prevent common fraud attempts, they often depend on fixed rules and manual intervention, making them less effective in identifying complex and continuously evolving fraud patterns. As digital payment transactions continue to increase, there is a growing need for intelligent machine learning–based solutions capable of analyzing large volumes of transaction data and adapting to new fraud techniques in real time.

2.2 Literature Analysis

Literature analysis on UPI fraud detection reveals that researchers and financial institutions have extensively explored the application of data mining, machine learning, and artificial intelligence techniques to identify fraudulent transactions. Various studies have focused on analyzing transaction patterns, user behavior, and historical financial data to improve fraud detection accuracy and reduce financial losses. Researchers have proposed several predictive models, including Decision Trees, Random Forest, Logistic Regression, Support Vector Machines, and Neural Networks, to classify transactions as genuine or fraudulent. These approaches aim to enhance security by automatically identifying suspicious activities and reducing dependence on manual monitoring. Several research works have demonstrated that machine learning–based fraud detection systems can achieve higher

accuracy and faster response times compared to traditional rule-based methods. However, challenges such as imbalanced datasets, evolving fraud techniques, high false-positive rates, and real-time processing requirements continue to affect system performance. The findings from existing literature indicate that intelligent machine learning models provide a promising solution for improving digital payment security and can significantly contribute to the development of efficient and reliable UPI fraud detection systems.

2.3 Comparative Study

The comparative analysis between existing fraud detection systems and the proposed project demonstrates significant improvements in fraud prediction capability, security, and operational efficiency. Existing systems generally depend on predefined rules, transaction thresholds, manual monitoring, and historical verification processes, which often produce delayed or inaccurate fraud detection results. These systems typically consider a limited set of transaction parameters and rely on fixed detection mechanisms that may not effectively adapt to rapidly evolving fraud techniques. In contrast, the proposed system introduces a Machine Learning–based approach that predicts fraudulent transactions by analyzing multiple transaction features such as transaction amount, transaction frequency, transaction time, location, device information, and user behavior patterns. Unlike traditional systems that depend on static rule sets, the proposed solution utilizes intelligent machine learning algorithms to learn from historical transaction data and identify complex fraud patterns automatically. Additionally, the system improves transparency through analytical dashboards, fraud statistics, and transaction monitoring features, enabling administrators to better understand system performance and fraud trends. The Django-based web application provides an interactive interface for user registration, authentication, transaction analysis, and fraud prediction. Through intelligent data analysis and automated decision-making, the proposed system achieves higher detection accuracy, better adaptability, improved reliability, and enhanced user security compared to existing rule-based fraud detection solutions.

CHAPTER – 3

SYSTEM ANALYSIS

EXISTINGSYSTEM:

Existing UPI fraud detection systems primarily depend on traditional rule-based mechanisms and security monitoring solutions implemented by banks and digital payment service providers. These systems identify fraudulent transactions using predefined rules, transaction limits, location verification, device authentication, and historical transaction records. Most existing solutions use limited transaction attributes such as transaction amount, transaction frequency, login activity, and account behavior to generate fraud alerts. Traditional fraud detection methods rely heavily on fixed rules and manual investigation, making fraud identification less adaptive and often inconsistent across different payment platforms. Some recent systems incorporate basic Machine Learning techniques to improve detection accuracy. However, these implementations generally depend on simple prediction models and limited data analysis capabilities. Existing platforms usually provide basic monitoring features with minimal analytical support and limited explanation of how fraud predictions are generated. Since fraudulent activities continuously evolve and involve complex transaction patterns, these approaches often fail to adapt quickly to new fraud techniques. Current systems also lack comprehensive model comparison, advanced behavioral analysis, and intelligent feature evaluation. As a result, detection accuracy, scalability, and real-time fraud prevention remain major challenges in existing digital payment security solutions.

LIMITATIONS OF EXISTING SYSTEM:

1. Existing systems rely heavily on predefined rules and manual fraud investigation methods.
2. Fraud detection often depends on limited transaction parameters and historical records.
3. Rule-based approaches cannot effectively identify complex and evolving fraud patterns.
4. Most systems consider only a limited number of transaction features.
5. Traditional methods may generate high false-positive and false-negative results.
6. Existing platforms have limited adaptability to emerging fraud techniques.
7. Many solutions depend on a single detection mechanism without comparing multiple algorithms.
8. Basic prediction techniques often fail to capture complex user behavior patterns.

9. Most existing applications provide limited transparency regarding fraud prediction decisions.
10. Real-time fraud detection and advanced analytical support are often absent or minimal.

PROPOSED SYSTEM

The proposed system introduces an intelligent Machine Learning–based web application designed to improve the accuracy and efficiency of UPI fraud detection. The system utilizes structured transaction data containing features such as transaction amount, transaction frequency, transaction time, location, device information, and user behavior patterns for predictive analysis. Multiple machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine (SVM) can be implemented and evaluated to identify the most effective fraud detection model. The application is developed using the Django framework and provides an interactive interface for user registration, authentication, transaction monitoring, and fraud prediction. The system workflow begins with transaction data collection, followed by preprocessing and feature engineering to prepare the dataset for model training. The selected model analyzes transaction details and predicts whether a transaction is genuine or fraudulent. The prediction results are displayed through dashboards and analytical reports. The system also supports efficient management of user and transaction records while providing fraud statistics and monitoring features. This integrated approach delivers improved fraud detection performance, enhanced security, and better decision support compared to existing rule-based systems.

ADVANTAGESOF PROPOSED SYSTEM

- Integrates Machine Learning with a responsive Django web application for intelligent fraud detection.
- Uses multiple predictive algorithms to improve fraud detection accuracy.
- Enables comparison among different machine learning models to select the best-performing algorithm.
- Machine learning techniques effectively handle complex and evolving fraud patterns.
- Provides interactive dashboards and analytical reports for better visualization of fraud detection results.
- Supports faster and more accurate identification of suspicious transactions.
- Enhances transaction security, reliability, and user confidence in UPI-based digital payment systems.

4.1 Hardware setup

Hardware Component	Requirement
Processor	Intel Core i3 or Above
RAM	4 GB Minimum
Storage	128 GB HDD/SSD
Monitor	Standard Display Monitor
Internet	Broadband / Mobile Hotspot

Table 4.1 Hardware Setup table

4.2 Software setup

Software Component	Requirement
Operating System	Windows 11
Programming Language	Python 3.11.9
Framework	Django
Frontend	HTML, CSS, JavaScript, Bootstrap
Database	MySQL 8.0
IDE	Visual Studio Code
Browser	Google Chrome
Design Software	Lucidchart

Table 4.2 Software Setup table

CHAPTER – 4

SYSTEM DESIGN

4.1 ARCHITECTURE DESIGN :

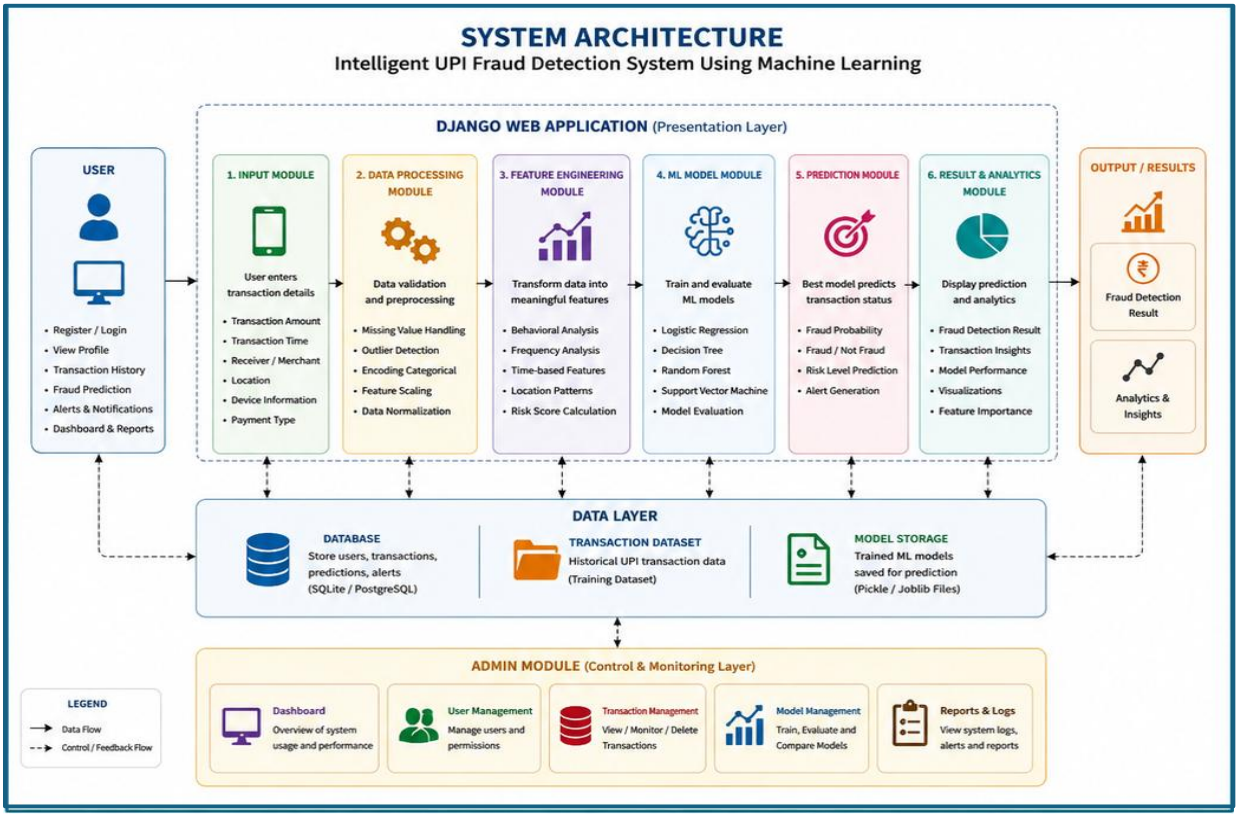


Figure 4.1: System Architecture For Intelligent UPI Fraud Detection System

The proposed Intelligent UPI Fraud Detection System Using Machine Learning illustrates the overall workflow of the application. The system begins with user transaction data input through the web interface, followed by data preprocessing and feature engineering to prepare the data for analysis. The processed data is then passed to the Machine Learning module, where fraud detection algorithms analyze transaction patterns and predict whether a transaction is genuine or fraudulent. The prediction results are displayed through dashboards and reports, while the database stores user information, transaction records, and prediction outcomes. The Admin module manages users, monitors fraud activities, and generates analytical reports for effective system management and decision-making. The architecture also supports secure user authentication, transaction monitoring, and efficient data handling. This integrated framework enhances fraud detection accuracy, improves transaction security, and provides reliable support for digital payment systems.

4.2 UML Models

Unified Modeling Language (UML) diagrams are used to represent the structural and behavioral design of the proposed system. In this project, UML modeling provides a visual representation of how users interact with the application, how data flows between modules, and how different system components collaborate to generate UPI fraud detection. The developed system follows a layered architecture integrating a Django web application, Machine Learning models, SQLite database, analytics dashboards, and administrative controls. UML diagrams help explain the complete functionality of the system before implementation and improve understanding of software design, workflow, and system interactions.

4.2.1 Use Case Diagram

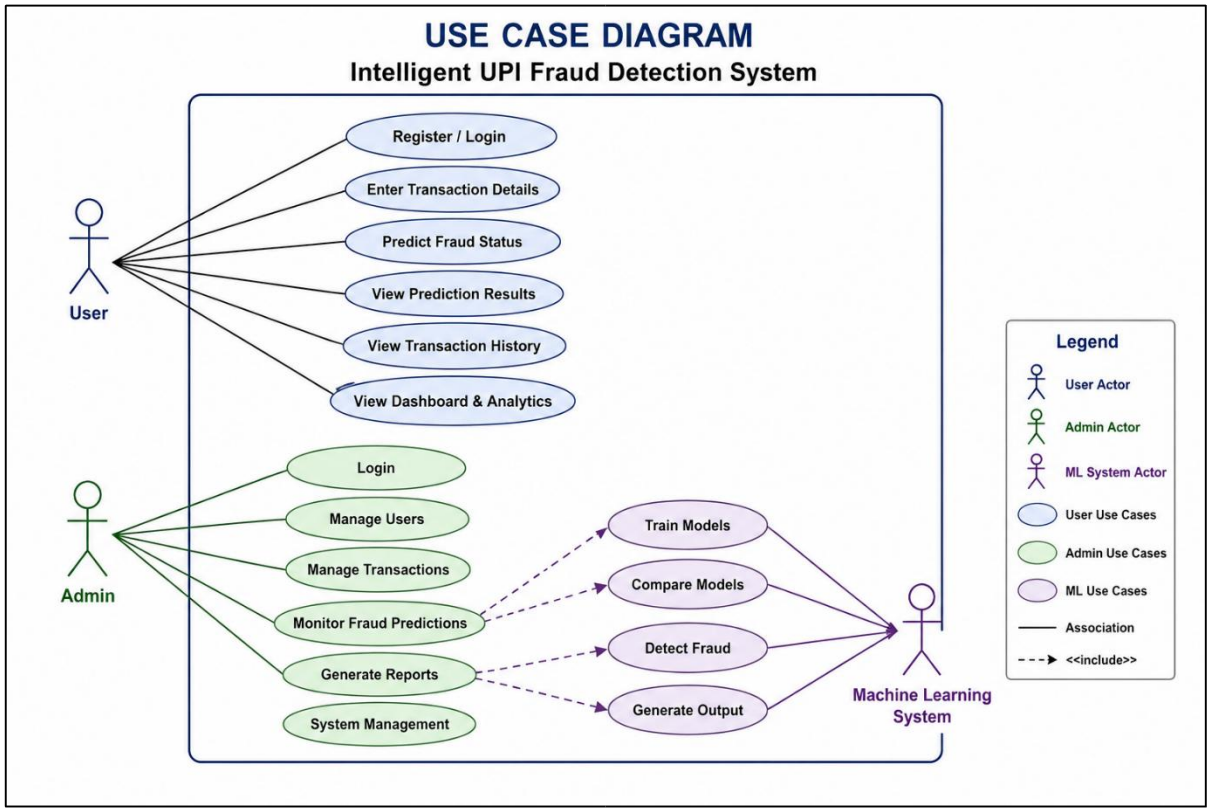


Figure 4.2.1: Use Case Diagram For UPI Fraud Detection

The Use Case Diagram represents the functional behavior of the proposed Intelligent UPI Fraud Detection System Using Machine Learning system and illustrates how different actors interact with the application to perform various operations. In this model, three primary actors are involved: User, Admin, and the Machine Learning System.

4.2.2 Class Diagram

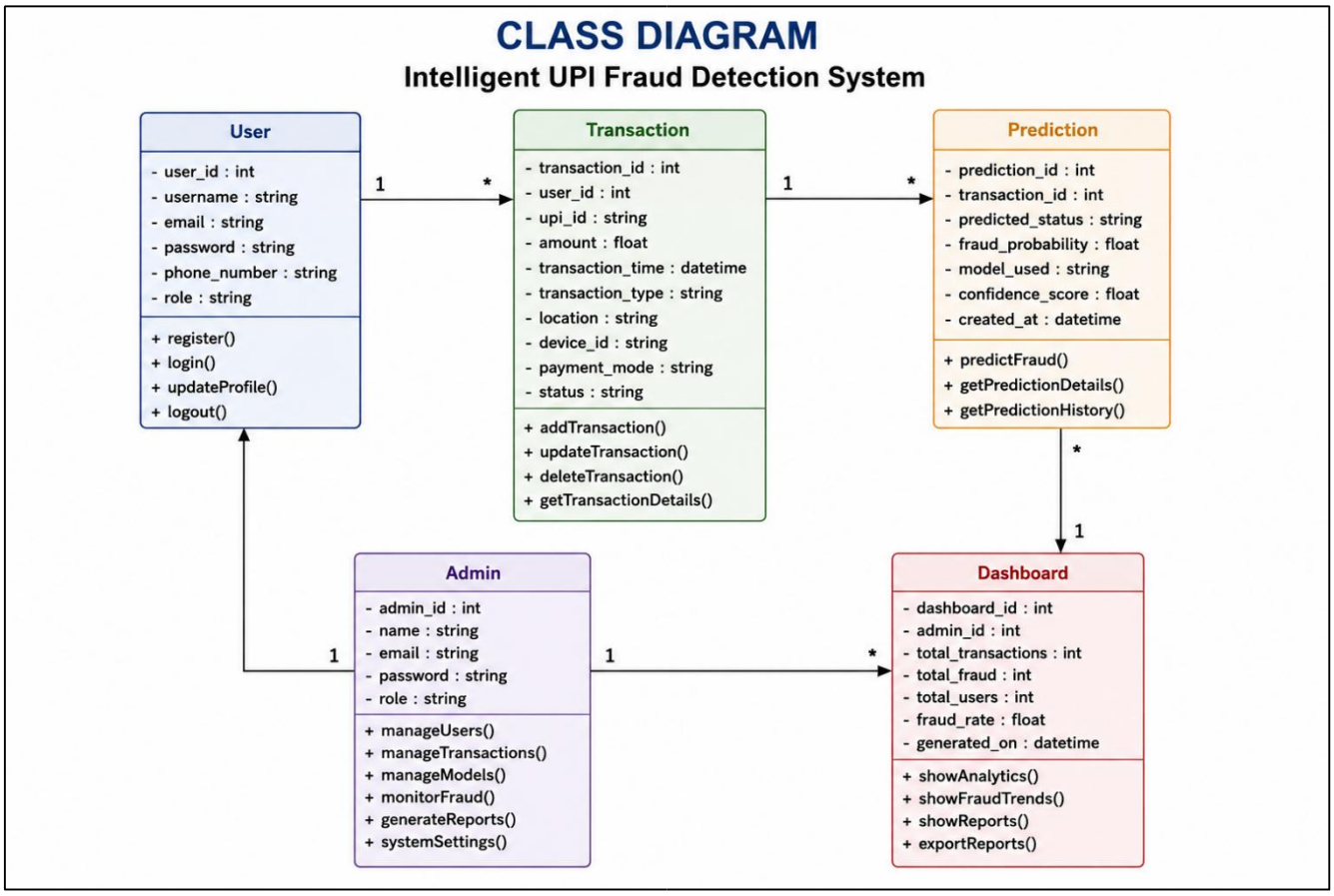


Figure 4.2.2 : Class Diagram For UPI Fraud Detection

The Class Diagram represents the structural design of the proposed Intelligent UPI Fraud Detection System Using Machine Learning and illustrates how different classes interact to perform fraud detection, transaction management, and analytics operations. It provides a static view of the system architecture by defining system entities, their attributes, methods, and relationships. The diagram consists of five major classes: User, Transaction Details, Fraud Prediction, Admin, and Dashboard, which collectively support the Django-based machine learning application. The Transaction Details class stores transaction-related information required for fraud detection. It includes attributes such as transaction_id, transaction_amount, transaction_time, location, device_information, transaction_type, and transaction_status. Methods including addTransaction(), updateTransaction(), deleteTransaction(), and getTransactionDetails() support management of transaction records. This class acts as the primary input source for machine learning prediction. The FraudPrediction class analyzes transaction data and generates fraud detection results, while the Dashboard class provides fraud statistics, reports, and analytical insights for effective monitoring and decision-making.

4.2.3 Sequence Diagram

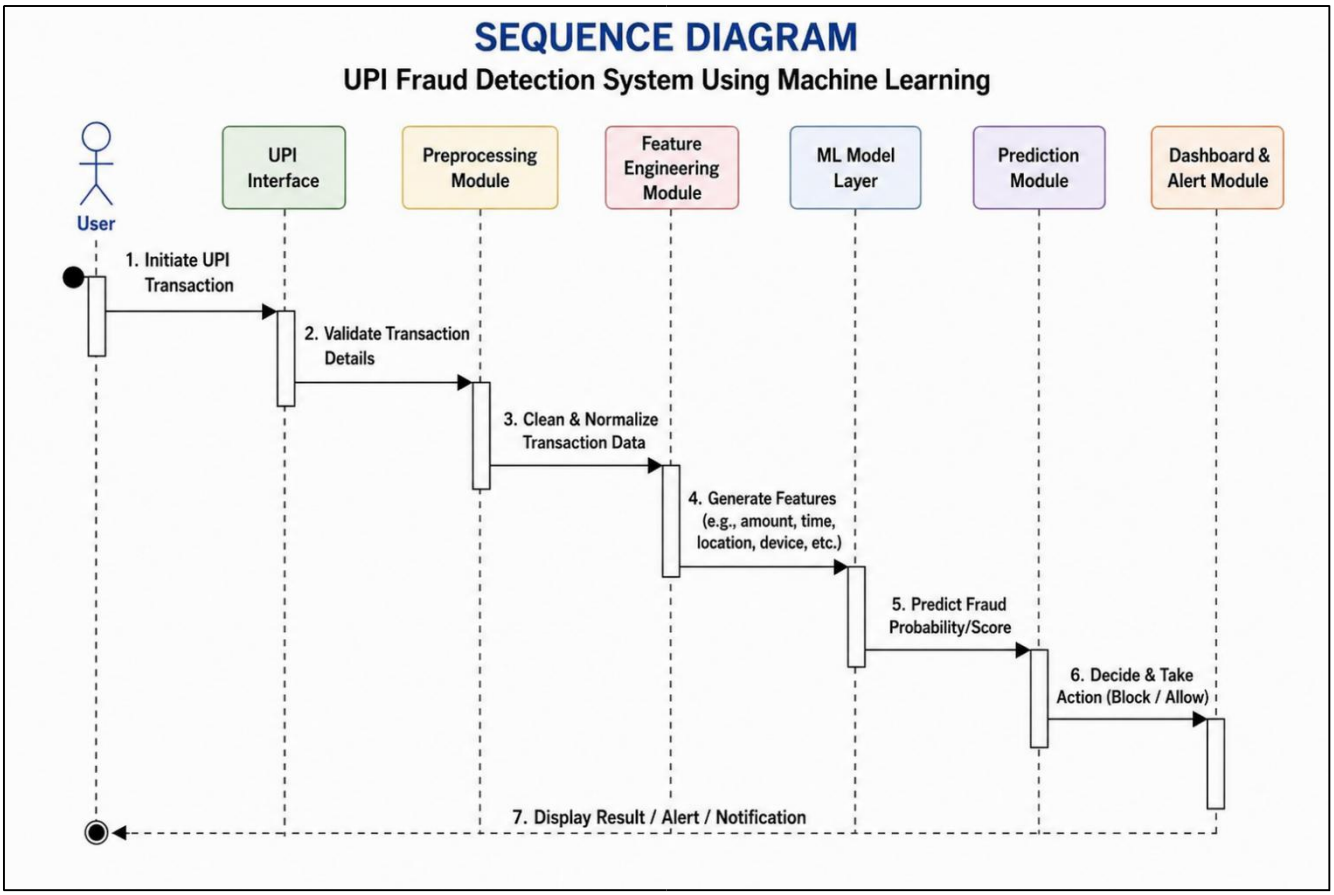


Figure 4.2.3 : Sequence Diagram For UPI Fraud Detection

The Sequence Diagram illustrates the interaction between different modules of the Intelligent UPI Fraud Detection System Using Machine Learning. The process begins when the user enters transaction details through the Django web interface. The transaction data is validated and forwarded to the preprocessing module, where data cleaning and transformation operations are performed. The feature engineering module extracts meaningful features from the processed transaction data and passes them to the Machine Learning layer. The ML model analyzes transaction patterns and forwards the results to the fraud prediction module, which determines whether the transaction is genuine or fraudulent. Finally, the prediction results, fraud statistics, and analytical insights are displayed on the dashboard for user and administrator review, enabling effective fraud monitoring and decision-making.

4.2.4 Collaboration Diagram

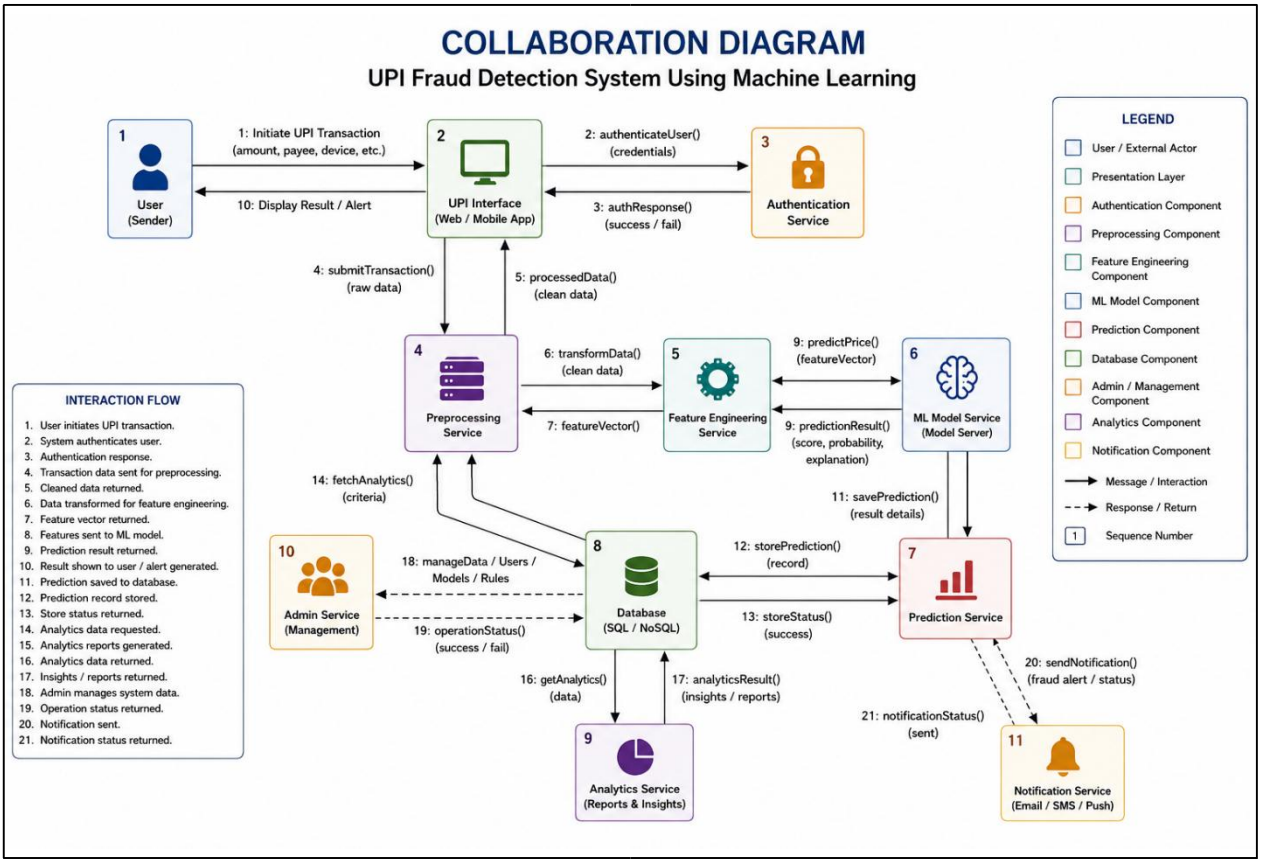


Figure 4.2.4 : Collaboration Diagram For UPI Fraud Detection

The Collaboration Diagram for the project “Intelligent UPI Fraud Detection System Using Machine Learning” illustrates how different system components interact and exchange information to identify fraudulent UPI transactions accurately and efficiently. The diagram focuses on communication between the User, Django Web Application, Preprocessing Module, Feature Engineering Module, Machine Learning Model Layer, Fraud Prediction Service, SQLite Database, Analytics Dashboard, and Admin Module. The workflow begins when the user enters transaction details through the Django interface. The application forwards this information to the preprocessing layer where validation, data cleaning, encoding, and transformation operations are performed. The processed data is then passed to the feature engineering module to generate meaningful predictive features. These features are sent to multiple machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine for fraud analysis and prediction. The fraud prediction service determines whether the transaction is genuine or fraudulent and stores the results in the SQLite (db.sqlite3) database. The analytics dashboard retrieves transaction records and generates reports, fraud statistics, and visual insights. Administrators can manage users, monitor transactions, analyze fraud activities, and maintain overall system performance.

4.2.5 Activity Diagram

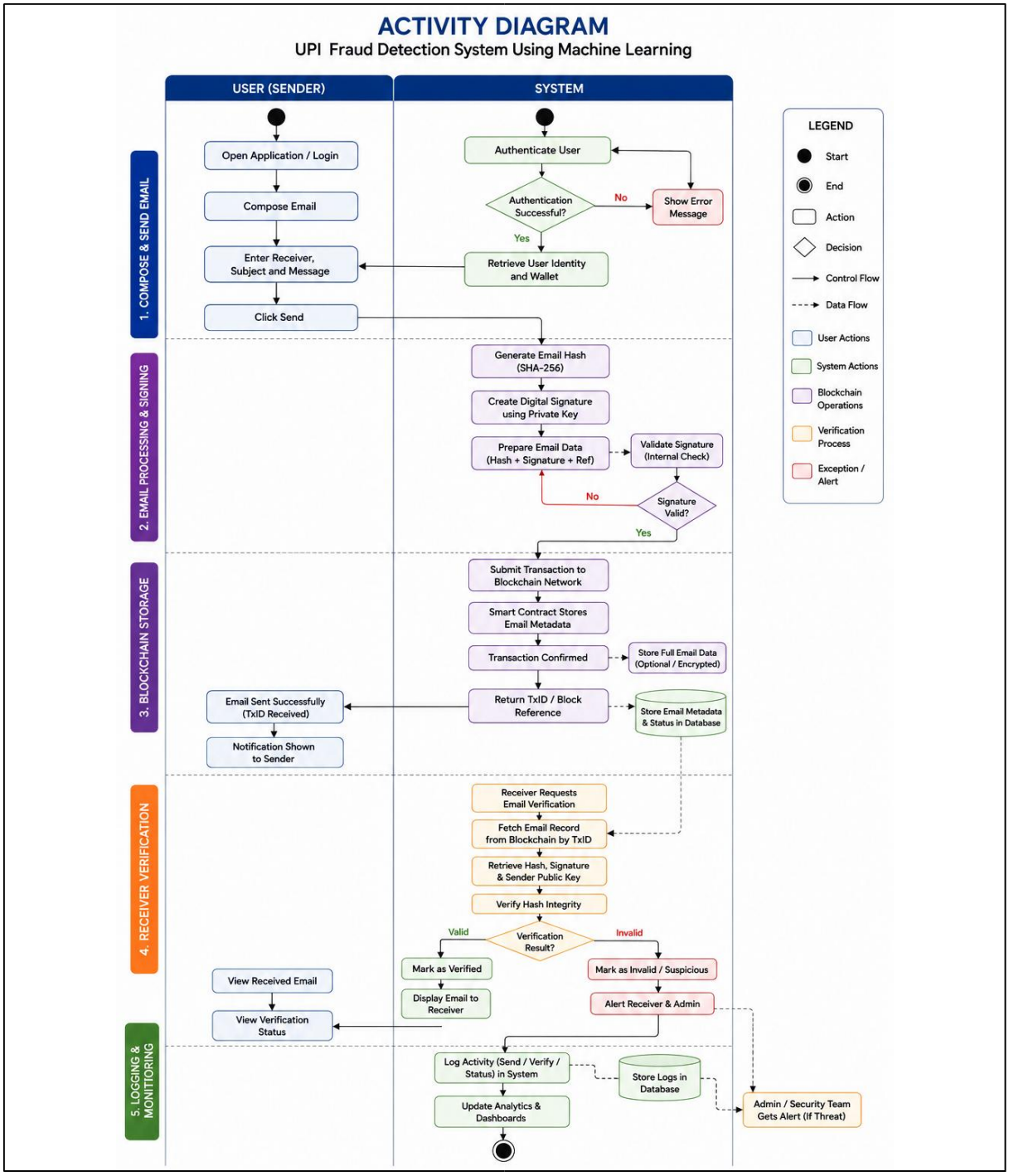


Figure 4.2.5 : Activity Diagram For UPI Fraud Detection

The Activity Diagram illustrates the workflow of the Intelligent UPI Fraud Detection System Using Machine Learning. The process begins when the user enters transaction details through the Django application. The system validates and preprocesses the data, applies machine learning algorithms for fraud analysis, generates prediction results, stores them in the database, and displays fraud detection outcomes through the dashboard.

4.2.6 Component Diagram

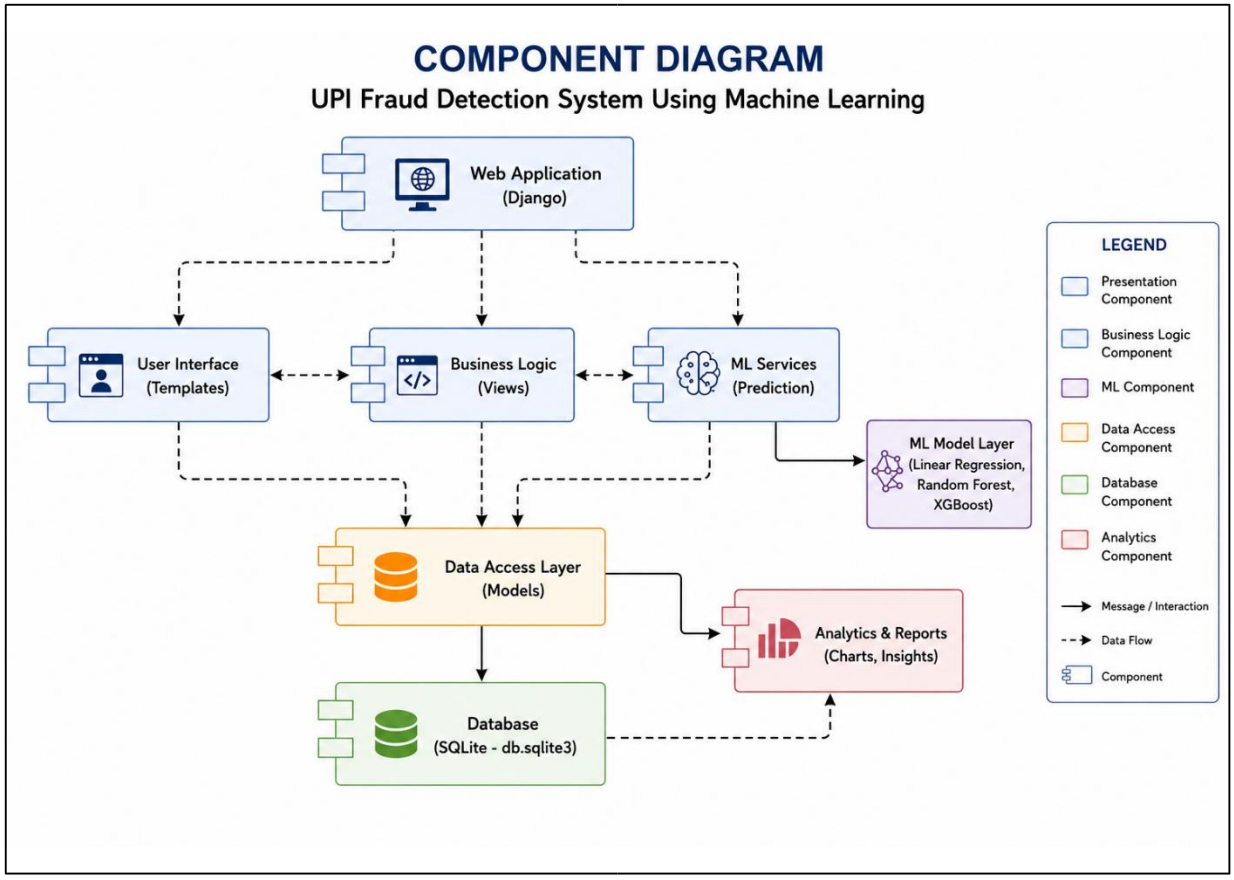


Figure 4.2.6 : Component Diagram For UPI Fraud Detection

The Component Diagram represents the internal structure and interaction of modules in the Intelligent UPI Fraud Detection System Using Machine Learning developed using Machine Learning and the Django Web Framework. It illustrates how different software components collaborate to collect transaction data, process information, perform fraud analysis, and display prediction results. The architecture begins with the Django Web Application, which acts as the central controller connecting all system modules. The User Interface provides pages for user registration, login, transaction submission, and result viewing. The Business Logic manages application operations and controls data flow between modules. The Data Access Layer stores and retrieves user and transaction records from the database. The Preprocessing and Feature Engineering modules prepare transaction data for analysis, while the Machine Learning module detects fraudulent transactions and generates prediction results for dashboard visualization and reporting.

4.2.7 Deployment Diagram

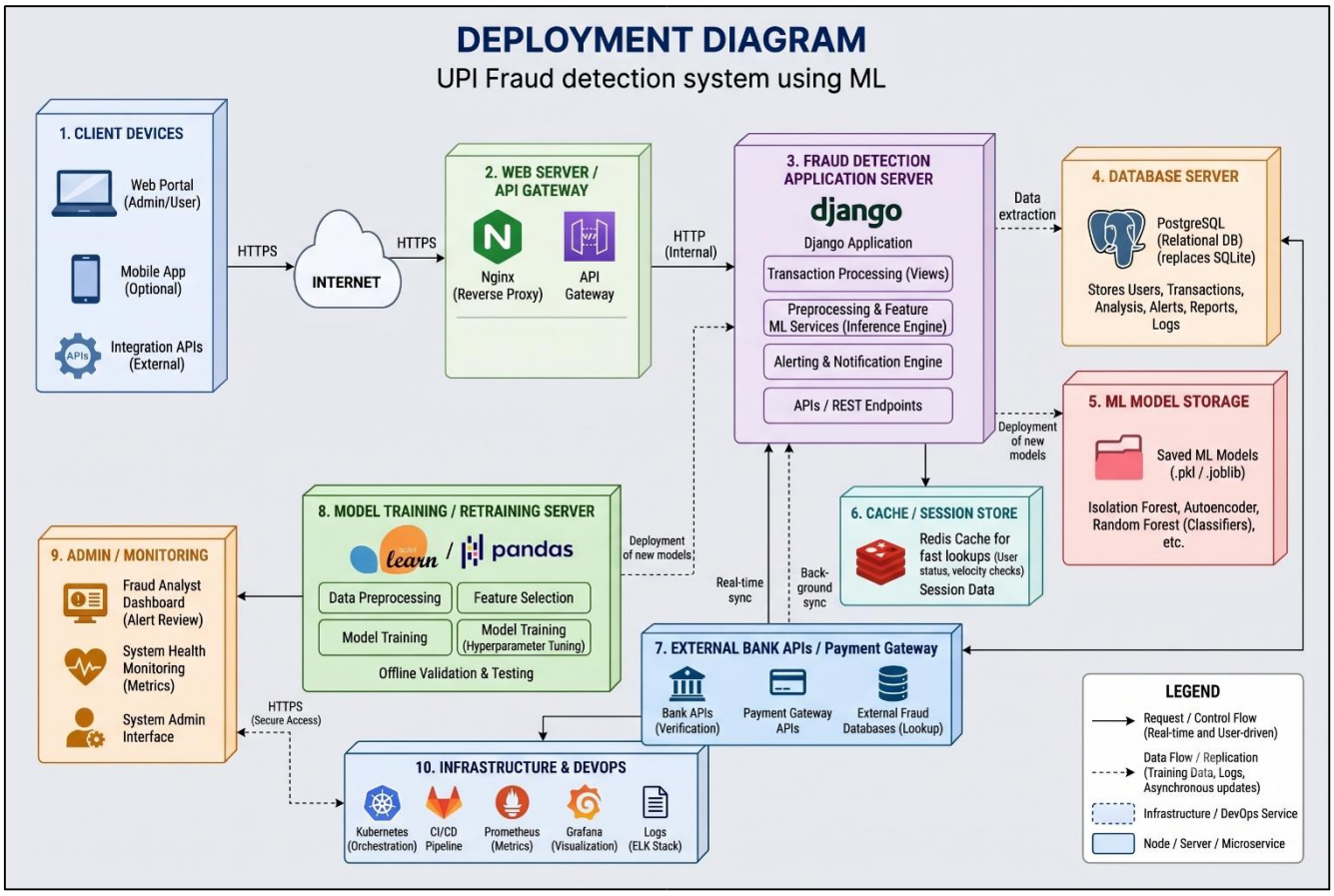


Figure 4.2.7 : Deployment Diagram For UPI Fraud Detection

The Deployment Diagram represents the physical deployment structure and execution environment of the Intelligent UPI Fraud Detection System Using Machine Learning project. It illustrates how the application components are deployed and communicate to provide accurate fraud detection and transaction monitoring. The architecture begins with Client Devices, where users access the system through a web browser to submit transaction details and view fraud prediction results. Requests are transmitted through the Internet to the Web Server, which manages incoming user requests and forwards them to the application layer. The requests are processed by the Application Server, where the Django Framework handles user authentication, transaction management, fraud prediction services, and dashboard generation. The application integrates Machine Learning modules that analyze transaction patterns and classify transactions as genuine or fraudulent. The system stores user information, transaction records, and prediction results in the SQLite Database, while the Admin module manages users, monitors fraud activities, and generates analytical reports for effective system administration.

4.3 Database Structure

The database structure of the proposed Intelligent UPI Fraud Detection System Using Machine Learning is designed to support efficient transaction management, fraud prediction, analytics generation, and administrative monitoring using SQLite (db.sqlite3) integrated with the Django web framework. The database acts as the central storage layer and enables communication between the user interface, machine learning models, and dashboard modules. The system stores user information and transaction-related data such as transaction amount, transaction time, location, device information, transaction type, and transaction status, which are used as input attributes for fraud prediction.

The database contains multiple interconnected tables to maintain structured application functionality. The User table stores user credentials and profile information and manages interactions within the system. The Transaction Details table records transaction attributes entered by users and serves as the primary dataset for fraud analysis. After data preprocessing and feature engineering, the processed data is passed to machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine, and the generated outputs are stored in the Fraud Prediction table. The ML Models table maintains model configurations, performance metrics, training details, and evaluation results. The Fraud Analytics table stores fraud statistics, prediction summaries, and transaction insights that support dashboard visualization and reporting.

Additional supporting tables such as Activity Logs, Monitoring Records, and Reports improve system transparency and administration. The Activity Logs table stores user activities and transaction history, while Monitoring Records track fraud detection events and system performance. Reports are generated for administrative analysis and documentation purposes. Overall, the database structure ensures efficient CRUD operations, reliable prediction storage, faster query execution, scalable machine learning integration, and seamless interaction between Django, fraud detection modules, dashboards, and SQLite database services. This structured design improves fraud detection accuracy, enhances maintainability, and supports future expansion of the system.

4.3.1 ER Diagram

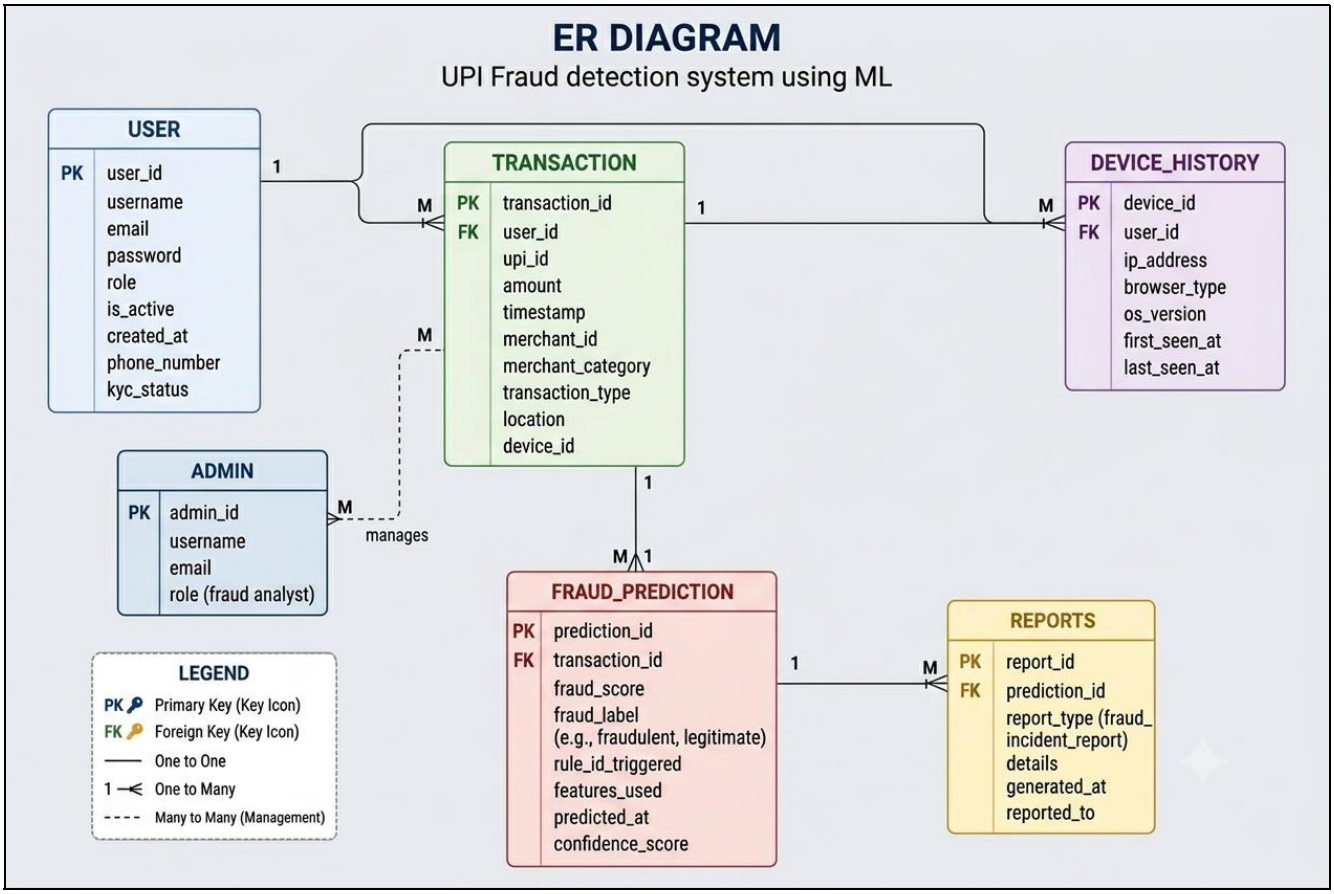
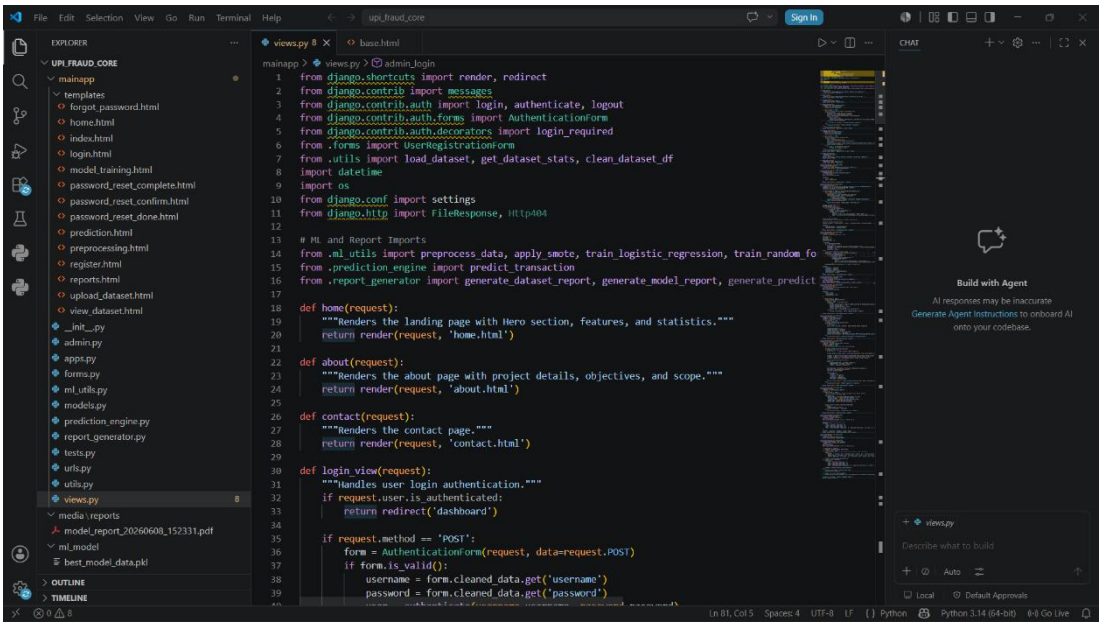


Figure 4.3.1 : ER Diagram For UPI Fraud Detection System

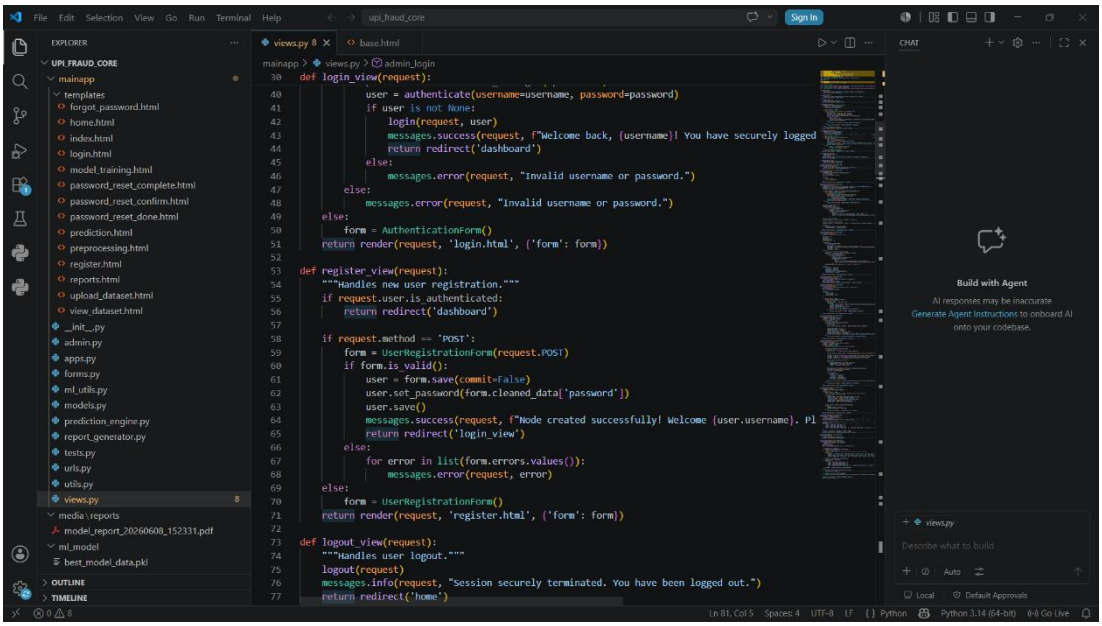
The ER (Entity Relationship) Diagram represents the database design of the Intelligent UPI Fraud Detection System Using Machine Learning developed using Django, Machine Learning, and SQLite (db.sqlite3). It illustrates how different entities such as User, Transaction Details, Fraud Prediction, Dashboard, Fraud Analytics, and ML Models are connected and interact within the system. Users provide transaction information which is stored in the database and processed through machine learning models to generate fraud prediction results. The prediction outputs, analytics, and reports are maintained for monitoring and visualization. The ER diagram helps define relationships between tables, ensures organized data storage, and supports efficient CRUD operations, fraud analysis, and system scalability.

4.3.2 SOURCE CODE



```python
from django.shortcuts import render, redirect
from django.contrib import messages
from django.contrib.auth import login, authenticate, logout
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth.decorators import login_required
from .forms import UserRegistrationForm
from .utils import load_dataset, get_dataset_stats, clean_dataset_df
import datetime
import os
from django.conf import settings
from django.http import FileResponse, Http404

# ML and Report Imports
from .ml_utils import preprocess_data, apply_smote, train_logistic_regression, train_random_fo
from .prediction_engine import predict_transaction
from .report_generator import generate_dataset_report, generate_model_report, generate_predict

def home(request):
    """Renders the landing page with Hero section, features, and statistics."""
    return render(request, 'home.html')

def about(request):
    """Renders the about page with project details, objectives, and scope."""
    return render(request, 'about.html')

def contact(request):
    """Renders the contact page."""
    return render(request, 'contact.html')

def login_view(request):
    """Handles user login authentication."""
    if request.user.is_authenticated:
        return redirect('dashboard')

    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            username = form.cleaned_data.get('username')
            password = form.cleaned_data.get('password')
```

Screenshot 4.3.2 : This source code contains the view functions, library imports, and request handlers that manage page rendering and application workflow.



```python
def login_view(request):
            user = authenticate(username=username, password=password)
            if user is not None:
                login(request, user)
                messages.success(request, f"Welcome back, {username}! You have securely logged
                return redirect('dashboard')
            else:
                messages.error(request, "Invalid username or password.")
        else:
            messages.error(request, "Invalid username or password.")
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

def register_view(request):
    """Handles new user registration."""
    if request.user.is_authenticated:
        return redirect('dashboard')

    if request.method == 'POST':
        form = UserRegistrationForm(request.POST)
        if form.is_valid():
            user = form.save(commit=False)
            user.set_password(form.cleaned_data['password'])
            user.save()
            messages.success(request, f"Node created successfully! Welcome {user.username}. Pl
            return redirect('login_view')
        else:
            for error in list(form.errors.values()):
                messages.error(request, error)
    else:
        form = UserRegistrationForm()
    return render(request, 'register.html', {'form': form})

def logout_view(request):
    """Handles user logout."""
    logout(request)
    messages.info(request, "Session securely terminated. You have been logged out.")
    return redirect('home')
```

Screenshot 4.3.2 : This source code implements the user registration functionality, including form validation, account creation, and profile management.

4.3.3 Data Dictionary

USERS

Field Name	Data Type	Description
user_id	UUID	Unique identifier for user
full_name	VARCHAR(100)	Full name of user
email	VARCHAR(150)	User email address
password_hash	TEXT	Encrypted password
created_at	DATETIME	Account creation time
status	VARCHAR(20)	User account status
Updated_at	DATETIME	Modification timestamp

TRANSACTION DETAILS

Field Name	Data Type	Description
transaction_id	UUID	Unique identifier for transaction
User_id	UUID	Unique identifier for use
Amount	Float	Transaction amount in INR
Transaction_count	INTEGER	Total transactions in past hour
Is_fraud	INTEGER	Fraud label (0 or 1)
Hour_of_day	INTEGER	Hour of transaction execution
Payment_mode	VARCHAR(30)	QR Code/UPI ID/Phone Number
Device_type	VARCHAR(30)	Android/iOS/Web
Status	VARCHAR(30)	Success/Failed/Pending
Created_at	DATETIME	Entry time

PREDICTION TABLE

Field Name	Data Type	Description
Prediction_id	INTEGER	Primary Key
Transaction_id	INTEGER	Foreign Key
Predicted_risk	FLOAT	Output value
Actual_label	FLOAT	Optional validation
Model_used	VARCHAR(50)	Selected model
Confidence_score	FLOAT	Prediction confidence
Prediction_time	DATETIME	Timestamp

CHAPTER – 5

IMPLEMENTATION

5.1 Algorithms Used in the System

The algorithms used in the proposed system are efficient, reliable, scalable, and suitable for accurate transaction fraud risk prediction. Each algorithm contributes to improving prediction accuracy, reducing estimation errors, and enhancing decision-making performance. The developed system applies multiple Machine Learning classification algorithms and compares their performance to identify the most suitable prediction model.

5.2 Logistic Regression Algorithm

Logistic Regression is used as the baseline prediction algorithm for estimating transaction fraud risks. It predicts values by establishing a logistic relationship between input transaction features and the probability of the output fraud class. The algorithm calculates coefficients for each feature and generates risk predictions based on mathematical classification equations.

5.3Random Forest Algorithm

Random Forest Classifier is used as the primary prediction model because of its high accuracy and strong generalization capability. This algorithm builds multiple decision trees and combines their outputs to generate final predictions.

Random Forest provides:

- Improved prediction accuracy
- Better handling of non-linear relationships
- Reduced overfitting
- Feature importance analysis
- Stable model performance

5.4XGBoostRegressorAlgorithm

XGBoost (Extreme Gradient Boosting) is implemented to improve prediction performance through optimized boosting techniques. The algorithm sequentially learns from previous errors and continuously improves model output. This model performs well for complex digital payment fraud patterns and improves overall system prediction quality.

5.5 Algorithm Comparison Table

Algorithm	Purpose	Advantages	Performance
Logistic Regression	Fraud Risk Estimation	Simple, Fast Training, Easy Interpretation	Moderate
Random Forest Classifier	Accurate Fraud Prediction	High Accuracy, Feature Importance, Reduced Overfitting	Very Good
XGBoost Classifier	Optimized Threat Prediction	High Scalability, Strong Generalization	Best
F1-Score Evaluation	Accuracy Measurement	Measures Model Reliability on Imbalanced Data	High
Precision Evaluation	False Positive Analysis	Minimizes Legitimate Transaction Blocking	Efficient
Recall Evaluation	Fraud Detection Sensitivity	Captures Maximum Actual Fraud Attacks	Reliable

Table 5.5.1 : Difference Algorithm Comparison Table Comparison Showing Performance

5.6 Main Modules

1. User Registration and Login Module
2. Admin Module
3. Input Module
4. Data Processing Module
5. Feature Engineering Module
6. Prediction Module
7. Machine Learning Module
8. Database Management Module

5.7 Module Explanation

1. User Registration and Login Module

The Registration and Login Module provides controlled access to the Intelligent UPI Fraud Detection System and manages user authentication within the Django web application. This module enables users to create an account, securely log in, and access personalized functionalities such as transaction fraud prediction, dashboard analytics, and prediction history. During registration, users provide details such as username, email address, and password, which are validated and stored in the SQLite (db.sqlite3) database. Django authentication mechanisms ensure secure credential handling and session management.

Main Functionalities:

- User Registration
- User Login Authentication
- Password Validation
- Session Management
- Logout Functionality
- Access Control for Users and Admin
- Secure Storage using SQLite Database
- Personalized Dashboard Access

2. Admin Module

The Admin Module provides monitoring and control functionalities for managing datasets, evaluating model performance, retraining models, and viewing prediction logs. It helps maintain system efficiency and supports future enhancements. Overall, these modules collectively enable efficient transaction data processing, intelligent fraud prediction, interactive visualization, and smooth application execution for accurate analysis of digital payment risks. Project reference considered from uploaded implementation structure and dashboard components.

3. Input Module

The Input Module is responsible for collecting transaction information from users through the Django web interface. Users enter required transaction details such as Amount, Transaction_count, Is_fraud, Hour_of_day, Payment_mode, Device_type, and Status. The module performs basic validation to ensure completeness and correctness of user inputs before sending them to further processing stages.

4. Data Processing Module

This module handles preprocessing of raw transaction data before model execution. It performs operations such as data cleaning, handling missing values, categorical encoding, feature scaling, and validation. The objective of this module is to transform raw input into a structured format suitable for Machine Learning algorithms.

5. Feature Engineering Module

The Feature Engineering Module extracts meaningful information from the processed dataset and prepares features for model training and prediction. Activities include feature transformation, feature selection, and train-test dataset preparation. This module improves prediction quality by selecting influential transaction attributes.

6. Prediction Module

The Prediction Module receives processed input data and forwards it to the trained Machine Learning model. The selected model computes and generates the estimated fraud risk of the transaction. This module provides fast and reliable valuation with reduced manual effort.

7.Machine Learning Module

This module forms the core intelligence of the system. Multiple classification algorithms are implemented and evaluated, including:

- Linear Regression
- Random Forest Regressor
- XGBoost Regressor

The models are trained using historical transaction datasets and evaluated based on prediction performance metrics. The best-performing model is selected for final prediction generation.

8. Database Management Module

The Database Module stores application-related information using SQLite (db.sqlite3). It manages transaction records, prediction history, model outputs, and system data. SQLite provides lightweight and efficient data management suitable for development and academic project deployment.

CHAPTER – 6

TESTING

6.1 Testing

Testing is an essential phase performed to verify that the developed system functions correctly and produces reliable prediction results. The Intelligent UPI Fraud Detection System was tested using multiple testing approaches including unit testing, integration testing, functional testing, performance testing, database testing, and user acceptance testing.

Testing was conducted on prediction modules, dataset processing, machine learning workflows, dashboard generation, registration and login functionality, database operations, and analytics visualization.

Types of Testing Performed

1. Unit Testing

Unit testing was conducted to validate individual modules independently. Modules such as registration, login, preprocessing, prediction generation, dashboard rendering, and model evaluation were tested separately to verify correctness and eliminate programming errors.

2. Integration Testing

Integration testing verified communication between multiple modules of the system. The Django application, Machine Learning layer, SQLite database, dashboard, and prediction engine were tested together to ensure smooth data exchange and proper workflow execution.

3. Functional Testing

Functional testing validated whether all system functionalities performed according to requirements. Features including user registration, transaction data entry, prediction generation, dashboard analytics, and administrative monitoring were tested successfully.

4. Performance Testing

Performance testing evaluated system speed, response time, prediction latency, and dashboard performance under different workloads. The application maintained stable prediction generation and smooth dashboard rendering.

5. Database Testing

Database testing verified storage and retrieval operations performed using SQLite (db.sqlite3). Transaction records, prediction history, user information, and model outputs were stored and retrieved successfully.

6. User Acceptance Testing

User Acceptance Testing was conducted to verify whether the developed system satisfies user requirements and project objectives. Test users successfully entered transaction details, generated predictions, accessed analytics dashboards, and reported positive usability experience.

Test Cases

Test Case Type	Test Case ID	Function	Expected Results	Actual Results	Low Priority	High Priority
Unit Testing	TC_001	User Registration	User account should be created successfully	User registered successfully	No	Yes
Unit Testing	TC_002	User Login Authentication	Valid user credentials should allow login	Login successful	No	Yes
Unit Testing	TC_003	Transaction Data Entry	Input Should Validate Correctly	Validation Successfully	Yes	No
Unit Testing	TC_004	Dataset preprocessing	Data should process correctly	Processing successfully	No	Yes

Unit Testing	TC_005	Fraud Risk Prediction	Predicted risk should generate	Prediction succesfully	No	Yes
Integration Testing	TC_006	Login with Dashboard	Dashboard should load after login	Dashboard loaded successfully	Yes	No
Integration Testing	TC_007	Predicition with ML Model	Prediction should execute correctly	Prediction successful	No	Yes
Integration Testing	TC_008	Prediction with Database	Records should store	Data stored successfully	No	Yes
Integration Testing	TC_009	Dashboard Analytics	Chart should display	Dashboard display	Yes	No
Functional Testing	TC_010	Predict fraud risk	Estimated value should appear	Result generated	No	Yes
Functional Testing	TC_011	Model comparision	Compare ML output	Comparision successfully	No	Yes
Functional Testing	TC_012	Analytics Dashboard	Analytics should display	Dashboard working	Yes	No
Functional Testing	TC_013	Admin Monitoring	Admin should monitor system	Monitoring successfully	Yes	No
Performance Testing	TC_014	Prediction Response time	Prediction should complete quick	Average 1.2 sec	Yes	No
Performance Testing	TC_015	Dashboard loading	Dashboard should render smoothly	Loaded successfully	Yes	No
Performance Testing	TC_016	Multiple Prediction Request	System should remain stable	Stable operation	No	Yes
Database Testing	TC_017	SQLite Record Storage	Prediction Should Store	Record stored	No	Yes

Database Testing	TC_018	Prediction History Retrieval	History Should Load	Retrieval successfully	Yes	No
User Acceptance Testing	TC_019	User Navigation	Navigation should be simple	Navigation smooth	Yes	No
User Acceptance Testing	TC_020	End-to-End Prediction	Users Should Obtain Accurate Results	Successfully	No	Yes

Table 6.1 : Difference Test Case Scenario For UPI Fraud Detection

6.1 Input Design

Input Design refers to the process of collecting, validating, and organizing user-provided data before it is processed by the Machine Learning models. In this project, the input module is designed using a responsive Django web interface where users enter transaction details required for predicting the fraudulent nature of digital transactions. The design focuses on simplicity, usability, and accurate data collection to improve prediction quality.

The system accepts structured inputs through form fields and validates them before sending them to the prediction engine. Users provide information related to transaction specifications including Amount Spent, Device ID, Transaction Frequency, Execution Time, Payment Mode, Device Type, and Transaction Status. Input validation ensures that incorrect or incomplete records are rejected before processing.

After submission, the data undergoes preprocessing such as missing value handling, categorical encoding, normalization, and feature transformation. The processed data is then converted into a machine-readable format and forwarded to the ML layer for prediction. Dropdown controls are used for categorical attributes to reduce manual errors and maintain consistency. Numeric fields enforce valid ranges and formats.

The input design supports efficient CRUD operations and enables future scalability for additional transaction attributes. The responsive interface ensures accessibility across different devices and provides a smooth interaction experience. Proper input handling minimizes data inconsistencies and improves overall system reliability and prediction accuracy.

INPUT PARAMETRE

- Amount
- Transaction_count
- Is_fraud
- Hour_of_day
- Payment_mode
- Device_type
- Status
- Submit Prediction Request

6.2 Output Design

Output Design defines how prediction results and analytical insights are presented to users after processing the input data. The output module of the proposed system is designed to provide meaningful, accurate, and visually understandable information through the Django-based dashboard interface.

After receiving user inputs and executing preprocessing and feature engineering, the trained Machine Learning models (Logistic Regression, Random Forest Classifier, and XGBoost Classifier) generate predicted fraud risks. The system compares model outputs and returns the most suitable prediction result.

The output interface displays the Predicted Risk Level, model performance metrics, and analytical visualizations for better interpretation. Dashboard components provide information such as prediction history, model evaluation statistics, feature importance analysis, and performance indicators including F1-Score, Precision, and Recall.

The output design emphasizes clarity, responsiveness, and transparency so users can understand how transaction characteristics influence predicted values. Interactive charts and reports improve usability and support decision-making. The generated outputs are stored in SQLite (db.sqlite3) for future analysis and reporting.

OUTPUT GENERATED :

- Predicted Fraud Risk
- Selected Best ML Model
- Prediction Accuracy Metrics
- Analytics Dashboard
- Reports and Logs
- Historical Prediction Records
- Model Performance Summary

CHAPTER – 7

RESULTS AND DISCUSSION

The Output Analysis phase is performed to evaluate the functionality, prediction accuracy, usability, reliability, and overall performance of the Intelligent UPI Fraud Detection System after implementation. This phase verifies whether the developed Machine Learning web application satisfies project objectives and provides accurate transaction valuation results. The system outputs include predicted transaction fraud risks, model evaluation reports, dashboard analytics, feature importance visualizations, historical prediction records, and administrative monitoring reports. These outputs are designed to improve transparency and support informed security decisions for users.

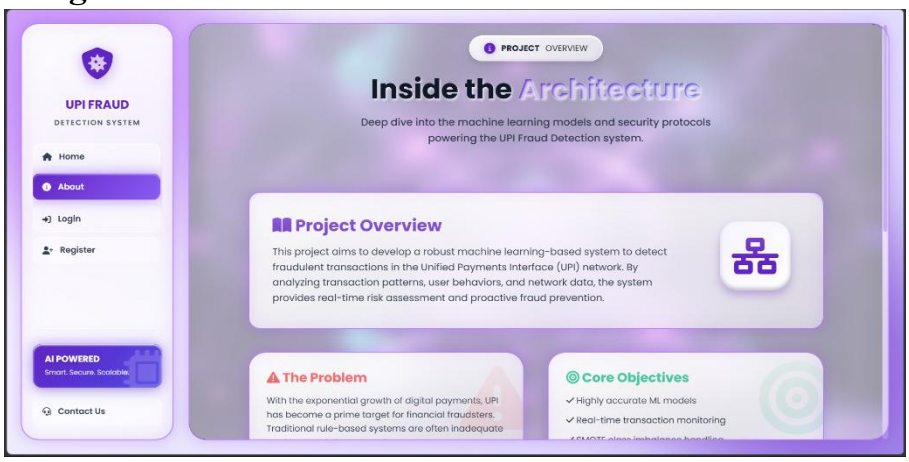
The prediction module generates estimated fraud risks based on transaction attributes such as Amount, Transaction_count, Is_fraud, Hour_of_day, Payment_mode, Device_type, and Status. Analytical outputs display model comparison results, prediction accuracy scores, feature importance graphs, and evaluation metrics including F1-Score, Precision, Recall, and Accuracy.

7.1 Screenshots

7.1.1 Home Page

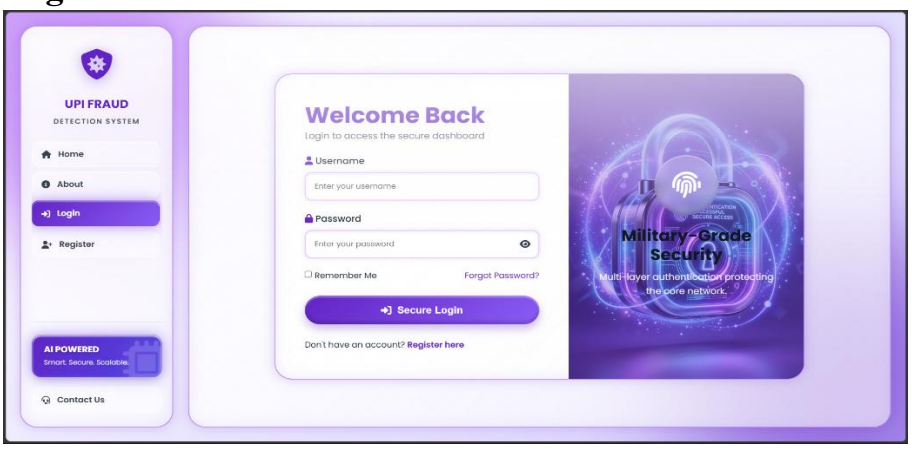
Output Screen 7.1.1 : user-friendly landing page that introduces the used car price prediction system

7.1.2 About Page



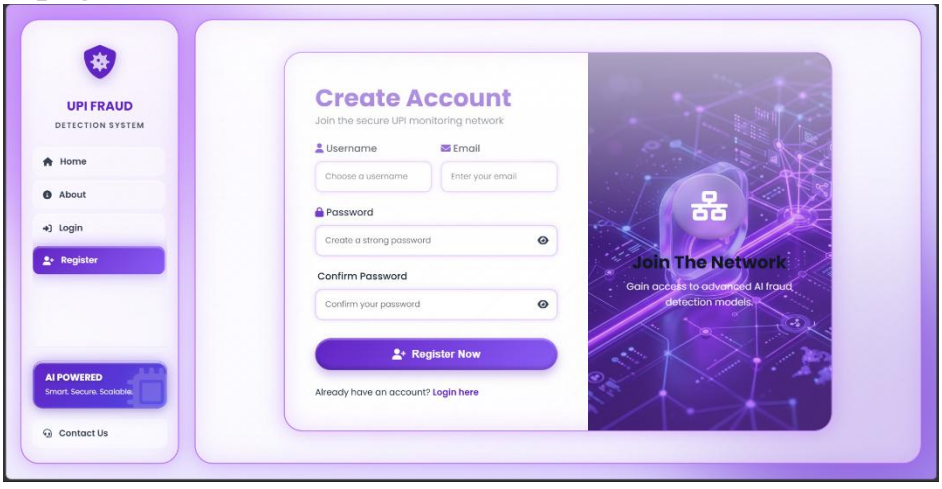
Output Screen 7.1.2 : About pages gives the project overview, the problem and core objectives of fraud detection system.

7.1.3 Login Page



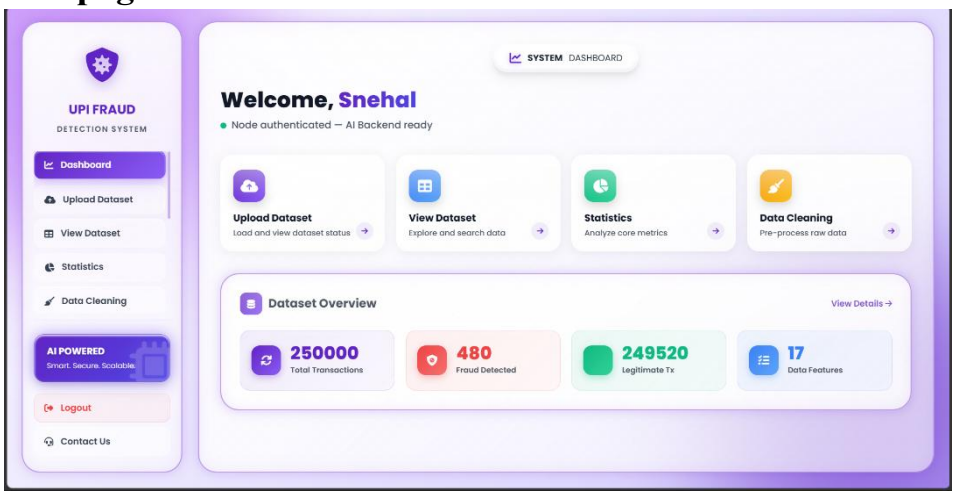
Output Screen 7.1.3 : A secure authentication page that enables registered users to access the system using their username and password.

7.1.4 Register page



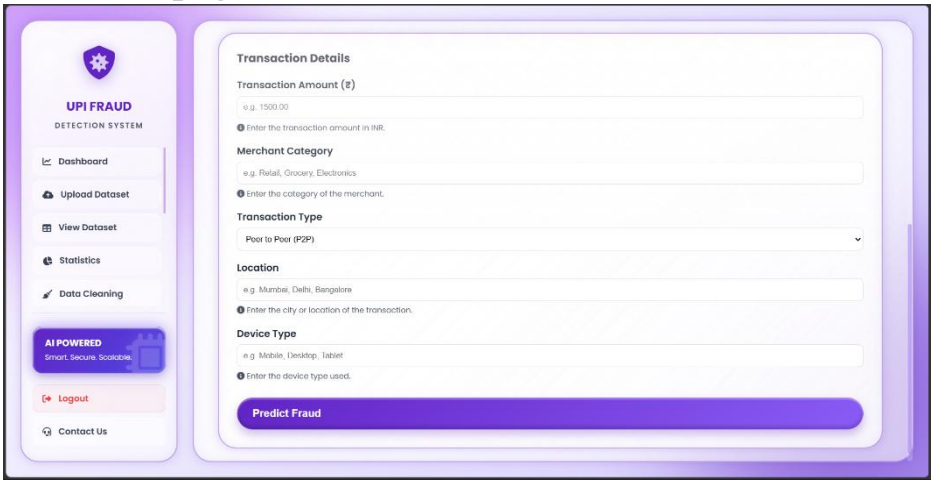
Output Screen 7.1.4 : user registration interface that allows new users to create an account by entering their personal details and login credentials.

7.1.5 Dashboard page



Output Screen 7.1.5 : user-friendly interface where users can enter vehicle details to generate an estimated used car selling price using machine learning.

7.1.6 Fraud Prediction page



Output Screen 7.1.6 : user-friendly interface where users can enter transaction details to predict fraud using machine learning.

CHAPTER – 8

CONCLUSION AND FUTURE SCOPE

The project “Intelligent UPI Fraud Detection System Using Machine Learning” was successfully designed and implemented to provide an intelligent and reliable financial security platform for predicting the fraudulent nature of digital transactions. The project mainly focused on solving challenges associated with traditional fraud detection methods such as manual verification, delayed response, limited scalability, and dependency on rule-based patterns. Machine Learning techniques were integrated with a robust-based web application to automate real-time risk assessment and improve prediction accuracy.

The developed system utilizes historical transaction listing data and applies multiple classification algorithms including Logistic Regression, Random Forest Classifier, and XGBoost Classifier to generate accurate fraud risk predictions. Data preprocessing, feature engineering, and model evaluation techniques were implemented to improve model performance and prediction reliability. The application uses transaction attributes such as Amount Spent, Device ID, Location, Time, Transaction Frequency, Merchant Type, and User Behavior to estimate transaction risk.

8.1 Project Outcome

The developed system successfully achieved the primary objective of predicting the fraudulent nature of digital transactions using Machine Learning techniques and web-based automation. Users can monitor transaction activities through an interactive interface and receive estimated fraud risks instantly.

The application processes historical transaction data and generates accurate predictions using multiple classification algorithms. Comparative model evaluation identified XGBoost Classifier as the most effective model for deployment due to its high accuracy and lower prediction error.

The dashboard module provides model analytics, prediction reports, performance monitoring, and feature importance visualization to improve transparency and decision-making. SQLite database integration enabled efficient storage and retrieval of prediction records and application data.

8.2 Future Enhancements

The proposed system can be further improved by integrating advanced Artificial Intelligence and Deep Learning techniques for more intelligent and adaptive transaction fraud prediction. Additional datasets containing global trends, regional anomalies, and real-time transaction streams can be incorporated to improve prediction accuracy.

Future versions may support real-time financial market data integration through external APIs for dynamic risk assessment. Advanced recommendation engines can be implemented to suggest optimal security settings and comparable fraud mitigation actions. Cloud deployment support can be introduced to improve scalability, availability, and performance in production environments. Mobile application support for Android and iOS platforms can enhance accessibility and user convenience.

Advanced analytics modules can be integrated for threat forecasting, fraud trend analysis, and interactive business intelligence dashboards. Future systems may also support user personalization, automated model retraining, and intelligent report generation.

CHAPTER – 9

REFERENCES

1.T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed., Springer, 2009.

2.A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow, 3rd ed., O'Reilly Media, 2022.

3.C. M. Bishop, Pattern Recognition and Machine Learning, Springer, 2006.

4.G. James, D. Witten, T. Hastie, and R. Tibshirani, An Introduction to Statistical Learning, Springer, 2021.

5.F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

6.D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed., John Wiley & Sons, 2013.

7.L. Breiman, "Random Forests," Machine Learning Journal, vol. 45, no. 1, pp. 5–32, 2001.

8.T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," Proceedings of ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

9.Django Software Foundation, Django Web Framework Documentation, Available: https://docs.djangoproject.com/

10. Python Software Foundation, Python Programming Language Documentation, Available: https://docs.python.org/

11. NumPy Developers, NumPy Documentation, Available: https://numpy.org/doc/

12.Pandas Development Team, Pandas Documentation, Available:
https://pandas.pydata.org/docs/

13.SQLite Development Team, SQLite Documentation, Available:
https://www.sqlite.org/docs.html

14.Kaggle Dataset Repository, UPI-Fraud-Detection Dataset,Available: https://www.kaggle.com/

15. UCI Machine Learning Repository, *Synthetic Financial Datasets for Fraud Detection*, Available: https://archive.ics.uci.edu/.